elastic newsroom

A2A and MCP: Better together!

Multi-Agent Architecture with A2A, MCP & Elasticsearch

View on GitHub

github.com/justincastilla/elastic-newsroom

Justin Castilla · April 2026

Agenda

1. **The Problem**
   Why multi-agent? · 3 min
1. **Architecture**
   A2A protocol & agents · 4 min
1. **MCP**
   Shared tools · 3 min
1. **Elasticsearch**
   Index, search, semantics · 4 min
1. **Live Workflow**
   Topic → article · 3 min
1. **Code & Lessons**
   Walkthrough · 3 min
1. **Q & A**
   5 min

The Problem

Single-LLM chatbots hit quality ceilings. No specialization. Research, writing, and editing are conflated into one step. No traceability of sources or editorial decisions.

The Solution

Five specialized AI agents, each with a clear role, coordinated by a “News Chief” via the A2A protocol.

5
Agents

8
MCP Tools

3
Search Modes

A2A · FastMCP · Tavily · Elasticsearch · Claude · React

Tech Stack

A2A Protocol
Google's
Agent-to-Agent SDK

OSS

FastMCP
Model Context Protocol
Shared tool registry

OSS

Tavily
Real-time web search
API
for research agents

API

Elasticsearch
Full-text, semantic &
hybrid vector search

FREE

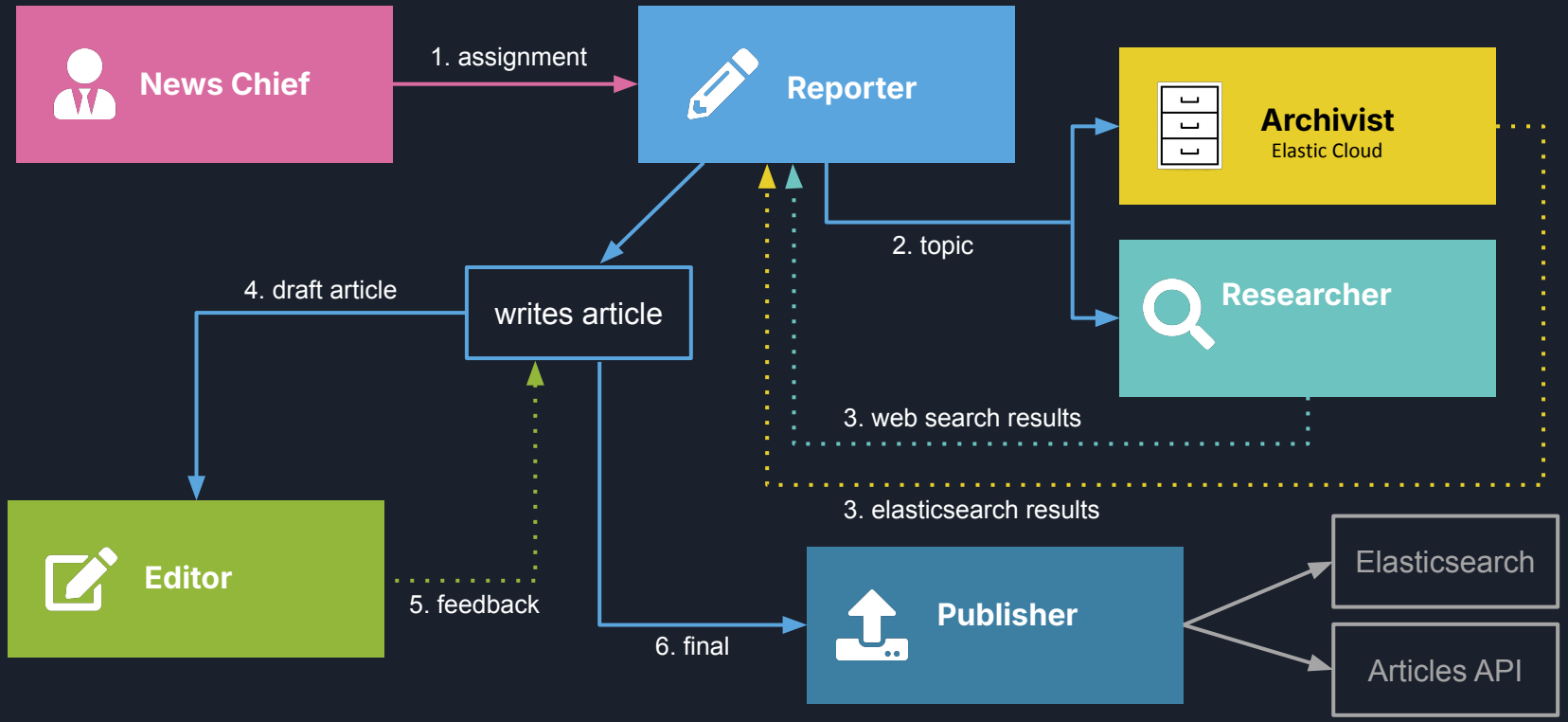
LLM API
Anthropic's Claude for
content generation

API

React + Starlette
Frontend UI & async
Python web framework

OSS

Architecture Overview



MCP Server · Event Hub (SSE) · Article API · React UI

A2A Protocol

Agent-to-Agent Communication

- **Agent Cards**

Self-describing JSON manifest with capabilities, skills, and transport preferences

- **JSON-RPC**

Standardized request/response between agents over HTTP

- **Task Model**

InMemoryTaskStore tracks state transitions across the workflow

- **Discovery**

Agents find each other by fetching `/.well-known/agent.json`

```
AgentCard(
 name = "Reporter",
 url = "http://localhost:8081",
 version = "1.0.0",
 capabilities = AgentCapabilities(
   streaming = False,
   state_transition_history = True,
   max_concurrent_tasks = 20
 ),
 skills = [
   AgentSkill(
     id = "write_article",
     name = "Write Article",
     description = "Create an article with a given topic"
     tags = ["writing", "research"]
   ),
   ...
 ]
)
```

A2A SDK — github.com/a2aproject/a2a-python

MCP — Model Context Protocol

A single FastMCP server exposes shared tools that any agent can call.

`generate_outline`

Structured story outline

`research_topic`

Tavily search + Claude synthesis

`write_article`

Full article from research

`review_article`

Grammar & tone review

`generate_tags`

AI-generated tags & categories

`extract_sources`

Source URLs from research

FastMCP — github.com/jlowin/fastmcp

```python
from fastmcp import FastMCP

mcp = FastMCP("newsroom-tools")

@mcp.tool()
async def research_topic(
    topic: str, questions: list[str],
    max_results: int = 5
) -> dict:
    """Research a topic using Tavily web search
    and synthesize findings with Claude."""

    # Search with Tavily
    results = await tavily_client.search(
        query=topic, max_results=max_results,
        search_depth="advanced"
    )

    # Synthesize with Claude
    synthesis = await anthropic.messages.create(
        model="claude-sonnet-4-6",
        messages=[{"role": "user",
                   "content": format_research(results)}]
    )
    return {"sources": results, "synthesis": synthesis}
```

@mcp Decorator

Well-written DocString

Business Logic

Elasticsearch — Indexing & Storage

31-field rich mapping

structured metadata with at-index actions, with nested objects, vector embeddings, and traditional field types

`semantic_text` fields

automatic vector embeddings without pipelines. BYOM or use default `jina-embeddings-v5-text`

`copy_to` attribute

automatically append source field into target field. This sets up the index for hybrid search

`nested` field type

field type for indexing arrays of objects where each object needs to maintain its field associations during queries

```
"mappings": {
  "properties": {
    "headline": {
      "type": "text",
      "copy_to": "headline_semantic"
    },
    "headline_semantic": {
      "type": "semantic_text"
    },
    "content": {
      "type": "text",
      "copy_to": "content_semantic"
    },
    "content_semantic": {
      "type": "semantic_text"
    },
    "research_sources": {
      "type": "nested",
      "properties": {
        "title": {"type": "text"},
        "url": {"type": "keyword"}
      }
```

Elasticsearch — Three Search Modes

Keyword

`multi_match` with field boosting and **`fuzziness`**

Semantic

Natural language via **`semantic_text`** with auto embeddings

Hybrid (RRF)

Combines both using Reciprocal Rank Fusion

```
response = es_client.search(
    index="news_archive",
    sub_searches=[
        {"query":
            {
              "multi_match": {
                "query": query,
                "fields": ["headline^3", "content", "topic^2"],
                "fuzziness": "AUTO"
        }}},
        {"query":
            {
              "semantic": {
                "field": "content_semantic",
                "query": query
        }}}
    ],
    rank={"rrf": {"window_size": 50, "rank_constant": 20}},
    size=limit
)
```

End-to-End Workflow

Real-time SSE events stream to React UI at every step — users watch the article being built live

```python
class NewsChiefAgent(BaseAgent):

    async def _assign_story(self, request):
        story_id = f"story_{datetime.now().strftime('%Y%m%d_%H%M%S')}"

        # Step 1: Send to Reporter via A2A
        reporter_result = await self._send_to_agent(
            agent_url="http://localhost:8081",
            message=json.dumps({
                "action": "write_article",
                "assignment": {"story_id": story_id,
                               "topic": topic}
            })
        )

        # Step 2: Send draft to Editor for review
        editor_result = await self._send_to_agent(
            agent_url="http://localhost:8082",
            message=json.dumps({
                "action": "review_draft",
                "draft": reporter_result["article"]
            })
        )

        # Step 3: Publish via Publisher on :8084
        await self._send_to_agent(
            PUBLISHER_URL, json.dumps({"action": "publish", ...})
        )
```

```python
class ReporterAgent(BaseAgent):
    async def _write_article(self, story_id, assignment):
        # Step 1: Generate outline and research questions
        outline, questions = await self._generate_outline(assignment)

        # Step 2: Query Researcher + Archivist in parallel
        research, archive = await asyncio.gather(
            self._send_to_agent(
                RESEARCHER_URL,
                {"action": "research", "questions": questions}
            ),
            self._send_to_archivist(
                ARCHIVIST_URL,
                {"topic": assignment["topic"]}
            )
        )

        # Step 3: Generate article from research + archive context
        article = await self._generate_article(
            assignment, outline, research, archive
        )
```

Lessons Learned

Specialization > Generalization

Five focused agents beat one do-everything LLM call

A2A makes agents composable

Swap any agent without touching the rest

MCP centralizes tool logic

One tool server provides consistent behavior, no duplication

`semantic_text` is magic

Elasticsearch handles embeddings automatically, no ingestion pipeline needed

SSE keeps users engaged and creates a logging trail

Watching the workflow live turns a wait into an experience and allows for forensics

Q & A

Justin Castilla

justincastilla@gmail.com

github.com/justincastilla/elastic-newsroom

A2A	FastMCP	Tavily
Elasticsearch	Claude	React